

```

#include <windows.h>

#include <GL/glut.h>

#include <cmath>

float angle = 0.0f;

float offsetX = 0.0f; // Horizontal oscillation

float offsetY = 0.0f; // Vertical movement

bool movingRight = true;

bool movingUp = true; // For vertical movement


// Function to draw a triangle

void drawTriangle() {

    glBegin(GL_TRIANGLES);

        glVertex2f(0.0f, 0.2f);

        glVertex2f(-0.2f, -0.2f);

        glVertex2f(0.2f, -0.2f);

    glEnd();

}


void display() {

    glClear(GL_COLOR_BUFFER_BIT);

```

```

// Triangle 1 - Rotating (Red)

glPushMatrix();

    glTranslatef(-0.5f, 0.5f, 0.0f);

    glRotatef(angle, 0.0f, 0.0f, 1.0f);

    glColor3f(1.0, 0.0, 0.0);

    drawTriangle();

glPopMatrix();


// Triangle 2 - Horizontal Oscillation (Green)

glPushMatrix();

    glTranslatef(offsetX, -0.4f, 0.0f);

    glColor3f(0.0, 1.0, 0.0);

    drawTriangle();

glPopMatrix();


// Triangle 3 - Static (Blue)

glPushMatrix();

    glTranslatef(0.5f, -0.5f, 0.0f);

    glColor3f(0.0, 0.0, 1.0);

    drawTriangle();

glPopMatrix();

```

```

// Triangle 4 - Vertical Movement (Yellow)

glPushMatrix();

    glTranslatef(0.0f, offsetY, 0.0f); // Moves vertically

    glColor3f(1.0, 1.0, 0.0);    // Yellow

    drawTriangle();

glPopMatrix();

glutSwapBuffers();
}

void update(int value) {

    angle += 1.0f;

    if (angle > 360.0f) angle -= 360.0f;

    // Horizontal oscillation (left-right)

    if (movingRight) {

        offsetX += 0.01f;

        if (offsetX > 0.5f) movingRight = false;

    } else {

        offsetX -= 0.01f;

        if (offsetX < -0.5f) movingRight = true;

    }
}

```

```

// Vertical movement (up-down)

if (movingUp) {

    offsetY += 0.01f;

    if (offsetY > 0.8f) movingUp = false; // Top boundary

} else {

    offsetY -= 0.01f;

    if (offsetY < -0.8f) movingUp = true; // Bottom boundary

}

glutPostRedisplay();

glutTimerFunc(16, update, 0); // ~60 FPS

}

int main(int argc, char** argv) {

    glutInit(&argc, argv);

    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);

    glutInitWindowSize(600, 600);

    glutCreateWindow("Animated Triangles");

    glutDisplayFunc(display);

    glutTimerFunc(0, update, 0);

    glutMainLoop();

    return 0;

}

```